

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

ETTIway - Sistem de Navigare Interactivă Campus

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer***

**în domeniul *Electronică, Telecomunicații și Tehnologia Informației*
programul de studii *Rețele și Software de Telecomunicații***

Conducător științific

ș.l. dr. ing. Laurențiu BOICESCU

Absolvent

Fraticiu Vlad-Alexandru

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **FRĂTICIU O. Vlad-Alexandru, 444D**

1. Titlul temei: ETTIway. Platformă Web pentru Localizarea și Ghidarea Utilizatorilor în Campusul Universitar

2. Descrierea temei:

Proiectul va avea ca obiectiv dezvoltarea unei platforme digitale care va permite utilizatorilor să se orienteze cu ușurință în interiorul campusului universitar. Aplicația va oferi:

- Harta interactivă a campusului, cu posibilitatea de a explora clădirile din campus, sălile de laborator și cele de curs
- Facilități de editare a hărții interactive, pentru administratorii platformei
- Facilități de localizare a utilizatorului, determinată prin sistemul de poziționare al dispozitivului.
- Căutarea rapidă a sălilor, clădirilor și laboratoarelor, cu afișarea informațiilor aferente.
- Rutare și ghidare către o destinație, prin calcularea traseului și afișarea acestuia în timp real.

Proiectul va contribui la îmbunătățirea modului în care studenții, profesorii și vizitatorii vor interacționa cu spațiul universitar, oferind o soluție modernă, accesibilă și scalabilă pentru navigarea în campus.

3. Contribuția originală:

A. Modelarea campusului

- Identificarea și structurarea zonelor principale: clădiri, intrări, săli, coridoare, căi pietonale, intersecții.
- Construirea unei reprezentări a campusului sub forma unei rețele logice (graf) pentru rutare și analiză spațială.
- Definirea metadatelor asociate fiecărei locații (denumire, descriere, poziție, categorie).

B. Sistemul de localizare și afișare în timp real • Implementarea funcționalității de obținere și actualizare continuă a poziției utilizatorului.

- Afișarea poziției pe hartă și sincronizarea mișcării utilizatorului cu interfața.
- Gestionarea preciziei și a semnalelor de localizare, inclusiv situațiile de erori sau lipsă de semnal.

C. Modulul de căutare și afișare a informațiilor

- Crearea sistemului de căutare inteligentă a sălilor și clădirilor campusului.
- Afișarea detaliilor pentru destinațiile selectate și generarea opțiunilor de navigare.
- Realizarea unui mecanism de filtrare (clădire, etaj, funcție, tip sală).

D. Rutare și calculul traseului optim

- Realizarea unui algoritm de determinare a traseului optim între două puncte din campus, utilizând structura grafică internă.
- Afișarea vizuală a traseului pe hartă și actualizarea sa în timp real pe măsură ce utilizatorul se deplasează.
- Tratarea situațiilor speciale precum zone inaccesibile, lucrări, rute alternative.

E. Interfață grafică și servicii suport

- Proiectarea unui design intuitiv, adaptat pentru telefoane mobile.
- Proiectarea și implementarea bazei de date aferente
- Integrarea panourilor de informații, meniurilor, funcțiilor de control și feedback vizual.
- Optimizarea timpilor de încărcare și a performanței generale a aplicației.

4. Resurse și bibliografie:

[Madkour et al., 2017] – A Survey of Shortest-Path Algorithms. ResearchGate, 2017.
(https://www.researchgate.net/publication/316736456_A_Survey_of_ShortestPath_Algorithms DOI:
10.48550/arXiv.1705.02044)

[Chen, 2022] – Dijkstra's method with min-priority queue optimization. arXiv, 2022.
(<https://arxiv.org/pdf/2208.00312.pdf>)

[Luxen, 2011] – Real-time routing with OpenStreetMap data (DOI: 10.1145/2093973.2094062)

5. Data înregistrării temei: 2026-01-04 21:28:42

Conducător(i) lucrare,

Ș. L. Dr. Ing. Laurențiu BOICESCU

Student,

FRĂTICIU O. Vlad-Alexandru

Director departament,

Conf. dr. ing. Șerban OBREJA

Decan,

Prof. dr. ing. Mihnea UDREA

Cod Validare: **97d3a2f81f**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *ETTIway - Sistem de Navigare Interactivă Campus*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Rețele și Software de Telecomunicații* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Fraticiu Vlad-Alexandru

.....

Cuprins

Lista acronimelor	vii
Introducere	1
0.1. Contextul Proiectului	1
0.2. Motivația și Obiectivele	1
0.3. Metodologia și contribuția proprie	2
1. Fundamente teoretice și tehnologii utilizate	3
1.1. Sisteme de navigare și hărți digitale	3
1.1.1. Reprezentarea hărților prin dale (tiles)	4
1.2. Navigarea exterioară și navigarea interioară	4
1.2.1. Navigarea exterioară	4
1.2.2. Navigarea interioară	5
1.2.3. Integrarea celor două niveluri	5
1.3. Date geospațiale și modelarea informației geografice	5
1.3.1. Sisteme de coordonate și sisteme de referință spațială	6
1.3.2. Reprezentarea geometriilor: WKT și WKB	6
1.3.3. OpenStreetMap ca sursă de date	6
1.4. Poziționarea și localizarea utilizatorului	7
1.4.1. Sistemul de poziționare globală	7
1.4.2. Precizia și limitările localizării	8
1.5. Algoritmi de determinare a drumului minim	8
1.5.1. Modelarea problemei prin grafuri	8
1.5.2. Algoritmul lui Dijkstra	9
1.5.3. Performanță și optimizări	9
1.6. Interfețe pentru dispozitive mobile	9
1.7. Arhitecturi de aplicații web	10
1.7.1. Separarea frontend–backend	10
1.7.2. Servicii REST și formatul JSON	11
1.8. Tehnologii utilizate în implementare	11
1.8.1. Spring Boot pentru backend	11
1.8.2. Leaflet și JavaScript pentru frontend	12

1.8.3. Organizarea datelor și extensibilitatea	12
1.9. Concluziile capitolului	12
2. Arhitectura sistemului	14
2.1. Vedere de ansamblu asupra arhitecturii	14
2.2. Nivelul de servicii (backend)	15
2.2.1. Punctul de intrare al aplicației	15
2.2.2. Accesul la date prin JPA	15
2.3. Nivelul de date	16
2.3.1. Entitatea utilizator	16
2.3.2. Entitatea pentru datele hărții	17
2.4. Securitate: autentificare și autorizare	17
2.4.1. Criptarea parolelor	17
2.4.2. Configurarea regulilor de acces	18
2.4.3. Încărcarea utilizatorilor și inițializarea contului de administrator	18
2.5. Nivelul de prezentare (frontend)	19
2.5.1. Inițializarea hărții și delimitarea zonei	20
2.5.2. Construirea grafului de navigare	20
2.5.3. Calculul traseului prin algoritmul lui Dijkstra	21
2.6. Fluxul de comunicare între componente	22
2.7. Concluziile capitolului	22

Lista acronimelor

API = Application Programming Interface

OSM = OpenStreetMap

GPS = Global Positioning System

REST = Representational State Transfer

JSON = JavaScript Object Notation

HTTP = Hypertext Transfer Protocol

JPA = Java Persistence API

WKB = Well-Known Binary

WKT = Well-Known Text

SRS = Spatial Reference System

Introducere

0.1 Contextul Proiectului

Problema principală care apare frecvent într-un campus universitar este de a integra navigația exterioară cu cea interioară în corpuri, săli și laboratoare, într-un singur flux digital coerent. Plecând de la această problemă, lucrarea propune ETTIway, o platformă construită pentru a aduce la un loc navigarea exterioară cât și pe cea interioară într-o experiență unitară atât pentru studenți, cât și pentru personalul universitar. Obiectivul principal a fost dezvoltarea unei soluții capabile să ofere localizare contextuală, căutare rapidă a punctelor de interes și suport pentru rutare între locații relevante din campus.

Arhitectura tehnică îmbină un backend realizat cu Spring Boot, responsabil de expunerea serviciilor și gestionarea datelor, cu o interfață web interactivă construită cu JavaScript și Leaflet, orientată spre claritate vizuală și timp de răspuns redus. O contribuție practică importantă este integrarea stratului de date spațiale provenit din OpenStreetMap, care permite reprezentarea fidelă a infrastructurii campusului și extinderea facilă către noi zone. În completarea funcțiilor de bază, platforma include mecanisme de afișare contextuală a informațiilor despre clădiri și spații, precum și o bază pentru rutare inteligentă între puncte selectate. Din perspectiva operațională, soluția oferă și suport pentru administrarea datelor geografice, astfel încât actualizarea hărților, a denumirilor și a punctelor de interes să poată fi realizată rapid, cu impact minim asupra utilizatorului final.

0.2 Motivația și Obiectivele

Motivația principală a proiectului pornește din dificultățile reale întâlnite în orientarea zilnică din campus: identificarea rapidă a clădirilor, localizarea corectă a sălilor și parcurgerea eficientă a traseelor dintre puncte relevante. În absența unei soluții unificate, utilizatorii sunt nevoiți să combine surse multiple de informație, adesea incomplete sau necorelate între ele. Din această perspectivă, ETTIway urmărește să ofere o experiență digitală coerentă, care să reducă timpul de căutare și să îmbunătățească orientarea atât pentru utilizatorii noi, cât și pentru cei familiarizați cu infrastructura universitară.

Obiectivele urmărite în dezvoltarea platformei au fost definite pe două direcții complementare: utilitatea practică pentru utilizatorul final și mentenabilitatea tehnică a sistemului. Astfel, platforma a fost proiectată pentru a integra harta campusului cu date indoor/outdoor într-o interfață unitară, pentru a permite căutarea contextuală a punctelor de interes și pentru

a susține rutarea între locații relevante. În paralel, arhitectura aplicației a fost gândită astfel încât actualizarea datelor geografice și extinderea către noi clădiri sau zone să poată fi realizate rapid, fără modificări majore în fluxul de utilizare.

0.3 Metodologia și contribuția proprie

Metodologia de dezvoltare a fost structurată în etape succesive, astfel încât fiecare componentă a sistemului să poată fi validată înainte de integrarea finală. În prima etapă a fost realizată analiza cerințelor funcționale și nefuncționale, cu accent pe scenariile reale de utilizare: localizarea rapidă a unei clădiri, identificarea unei săli și parcurgerea unui traseu relevant în campus. Pe baza acestei analize a fost definită arhitectura aplicației, separată pe nivelul de prezentare (frontend), nivelul serviciilor (backend) și nivelul datelor geografice. Implementarea a fost realizată incremental, prin dezvoltarea și testarea separată a modulelor de vizualizare, căutare și gestiune a datelor indoor, urmată de integrarea lor într-un flux unitar.

Contribuția proprie vizează atât partea tehnică, cât și partea de organizare a datelor spațiale. La nivel de date, contribuția include structurarea seturilor geospațiale utilizate de aplicație, adaptarea lor pentru scenariile de navigare și construirea unei baze extensibile pentru adăugarea de noi clădiri, niveluri și elemente de infrastructură. Rezultatul este o soluție funcțională, ușor de extins, orientată spre utilizare practică în mediul universitar.

Capitolul 1

Fundamente teoretice și tehnologii utilizate

Acest capitol prezintă fundamentul teoretic și tehnologic pe care se sprijină platforma ETTIway. Sunt abordate, în mod gradual, conceptele de bază ale sistemelor de navigare digitală, particularitățile navigării în spații exterioare și interioare, modul de reprezentare a datelor geospațiale, precum și tehnologiile concrete folosite în implementare. Scopul acestui capitol este de a oferi cititorului contextul necesar pentru a înțelege deciziile de proiectare descrise în capitolele următoare, plecând de la principiile generale și ajungând la instrumentele specifice care au stat la baza dezvoltării aplicației.

1.1 Sisteme de navigare și hărți digitale

Un sistem de navigare digitală reprezintă un ansamblu de componente software și de date care permite unui utilizator să se localizeze într-un spațiu, să identifice puncte de interes și să determine trasee între acestea. Spre deosebire de hărțile tipărite, hărțile digitale sunt interactive: utilizatorul poate mări sau micșora nivelul de detaliu, poate filtra informația afișată și poate interoga elementele de pe hartă pentru a obține date suplimentare. Această interactivitate este esențială într-un mediu complex, cum este un campus universitar, unde densitatea informației relevante este ridicată, iar nevoile utilizatorilor variază considerabil de la o situație la alta.

În general, un sistem de navigare modern se construiește pe trei piloni fundamentali: un strat de date geografice care descrie spațiul fizic, un motor de procesare care interpretează aceste date și răspunde interogărilor, și o interfață de vizualizare care prezintă informația într-o formă accesibilă utilizatorului. Calitatea experienței finale depinde de modul în care aceste trei niveluri sunt corelate și optimizate. Un strat de date incomplet conduce la rezultate inexacte, un motor de procesare lent degradează experiența interactivă, iar o interfață confuză anulează beneficiile unei baze de date bine structurate.

Hărțile digitale interactive au cunoscut o dezvoltare accentuată odată cu apariția bibliotecilor de cartografie web, care permit afișarea hărților direct în browser, fără instalarea unor aplicații dedicate. Acest model, în care harta este servită ca o aplicație web, prezintă avantaje semnificative din punct de vedere al accesibilității: utilizatorul are nevoie doar de un browser modern și de o conexiune la internet, fără dependențe de sistem de operare sau de hardware specializat. ETTIway adoptă tocmai acest model, fiind o aplicație web ce poate fi accesată de pe orice dispozitiv cu un browser compatibil.

1.1.1 Reprezentarea hărților prin dale (tiles)

Una dintre tehnicile fundamentale folosite în cartografia web este împărțirea hărții în dale (în engleză *tiles*), imagini pătrate de dimensiune fixă, de regulă 256 pe 256 de pixeli, care acoperă o anumită zonă geografică la un anumit nivel de zoom. În loc să se încarce o singură imagine de dimensiune foarte mare, harta este compusă dinamic din mai multe asemenea dale, dintre care se afișează doar cele vizibile în fereastra curentă. Pe măsură ce utilizatorul se deplasează sau modifică nivelul de zoom, sunt încărcate noile dale necesare, iar cele care ies din câmpul vizual sunt eliberate din memorie.

Acest mecanism de afișare are mai multe avantaje practice. În primul rând, reduce volumul de date transferat la un moment dat, deoarece se transmit doar porțiunile vizibile ale hărții. În al doilea rând, permite stocarea în memoria cache a dalelor deja descărcate, ceea ce accelerează navigarea repetată prin aceeași zonă. În al treilea rând, sistemul de dale se pretează foarte bine la mai multe niveluri de detaliu: la niveluri mici de zoom se afișează contururi generale, iar la niveluri mari apar detalii fine, precum clădiri individuale, intrări sau trasee pietonale. Această organizare piramidală a datelor este folosită de majoritatea serviciilor de hărți web și constituie fundamentul pe care se construiește și experiența vizuală din ETTIway.

1.2 Navigarea exterioară și navigarea interioară

O distincție esențială în domeniul navigării este cea dintre navigarea exterioară (*outdoor*) și cea interioară (*indoor*). Cele două tipuri de navigare ridică probleme diferite atât din punct de vedere al achiziției datelor, cât și din punct de vedere al reprezentării și al modului de interacțiune cu utilizatorul. Una dintre contribuțiile centrale ale lucrării de față este tocmai integrarea acestor două dimensiuni într-un flux unitar, astfel încât utilizatorul să poată trece în mod natural de la nivelul campusului la nivelul unei clădiri și, mai departe, la nivelul unei săli sau al unui laborator.

1.2.1 Navigarea exterioară

Navigarea exterioară se referă la orientarea în spațiul deschis: identificarea clădirilor, a aleilor, a intrărilor și a traseelor dintre diferite puncte de interes ale campusului. La acest nivel, datele geografice sunt în general disponibile prin servicii cartografice publice și pot fi reprezentate în sisteme de coordonate geografice standard, exprimate prin latitudine și longitudine. Pozițiile sunt asociate unui sistem global de referință, ceea ce permite suprapunerea corectă a diferitelor straturi de informație și compatibilitatea cu datele provenite din surse externe.

Pentru un campus universitar, navigarea exterioară presupune reprezentarea fidelă a infrastructurii: conturul clădirilor, denumirile acestora, aleile pietonale, drumurile de acces și punctele de interes precum intrările principale, bibliotecile, cantinele sau punctele de orientare. Provocarea principală la acest nivel nu este atât tehnologică, cât legată de completitudinea și

acuratețea datelor: o hartă exterioară este utilă doar în măsura în care reflectă corect realitatea din teren și este actualizată atunci când infrastructura se modifică.

1.2.2 Navigarea interioară

Navigarea interioară se ocupă de orientarea în interiorul clădirilor: identificarea sălilor, a laboratoarelor, a etajelor și a traseelor dintre acestea. Acest tip de navigare ridică dificultăți suplimentare în comparație cu cea exterioară. Pe de o parte, datele de interior nu sunt, în general, disponibile în servicii cartografice publice și trebuie construite manual, plecând de la planuri ale clădirilor. Pe de altă parte, reprezentarea spațiului interior implică o componentă verticală — etajele — care nu poate fi exprimată printr-o simplă pereche de coordonate plane, ci necesită un mecanism de organizare pe niveluri.

În cadrul ETTIway, navigarea interioară este tratată prin asocierea fiecărei clădiri cu un set de planuri de etaj, fiecare plan corespunzând unui nivel distinct al clădirii. Utilizatorul poate selecta o clădire de pe harta exterioară și poate apoi naviga între etajele acesteia, vizualizând dispunerea sălilor și a spațiilor relevante. Această abordare, în care nivelul exterior și cel interior sunt corelate prin intermediul entității „clădire”, permite o tranziție coerentă între cele două perspective și constituie nucleul funcțional al platformei.

1.2.3 Integrarea celor două niveluri

Marea provocare a unui sistem complet de navigare în campus este unificarea celor două niveluri într-o singură experiență, fără discontinuități. În multe soluții existente, navigarea exterioară și cea interioară sunt tratate de aplicații separate sau de surse de date necorelate, ceea ce obligă utilizatorul să combine manual informații provenite din mai multe locuri. ETTIway propune o abordare integrată, în care o clădire selectată pe harta exterioară devine punctul de intrare către reprezentarea interioară a acesteia, păstrând astfel un fir narativ continuu pentru utilizator. Această integrare reprezintă una dintre contribuțiile practice principale ale lucrării și este detaliată în capitolele dedicate arhitecturii și implementării.

1.3 Date geospațiale și modelarea informației geografice

Fundamentul oricărui sistem de navigare îl constituie datele geospațiale, adică informația care descrie poziția și forma obiectelor în spațiu. Modul în care aceste date sunt structurate, stocate și interogate influențează direct performanța și flexibilitatea întregului sistem. În această secțiune sunt prezentate conceptele de bază necesare înțelegerii stratului de date al aplicației ETTIway.

1.3.1 Sisteme de coordonate și sisteme de referință spațială

Pentru a putea localiza un obiect pe suprafața Pământului este necesar un sistem de referință spațială (SRS), care definește modul în care coordonatele numerice se traduc în poziții reale. Cel mai răspândit sistem de referință folosit în aplicațiile web este cel bazat pe latitudine și longitudine, exprimate în grade, asociat unui model standardizat al Pământului. Acest sistem este utilizat pe scară largă deoarece este compatibil cu majoritatea surselor de date publice și cu serviciile de poziționare prin satelit.

Pe lângă sistemul geografic exprimat în grade, în cartografia web este frecvent folosită și o proiecție plană, care transformă suprafața curbă a Pământului într-un plan, facilitând afișarea hărților pe ecran. Trecerea între diferite sisteme de referință se numește reproiecție și trebuie tratată cu atenție, deoarece o suprapunere incorectă a straturilor de date conduce la afișarea eronată a obiectelor. În cadrul ETTIway, consecvența sistemului de referință între datele provenite din diverse surse este o condiție necesară pentru alinierea corectă a infrastructurii campusului.

1.3.2 Reprezentarea geometriilor: WKT și WKB

Obiectele geografice — puncte, linii și poligoane — sunt descrise prin geometrii. O clădire poate fi reprezentată ca un poligon, o alee ca o linie, iar un punct de interes ca un singur punct. Pentru a stoca și a transmite aceste geometrii există formate standardizate. Formatul textual WKT (*Well-Known Text*) descrie geometriile într-o formă lizibilă pentru om, în timp ce formatul binar WKB (*Well-Known Binary*) le codifică într-o reprezentare compactă, optimizată pentru stocare și transfer. Ambele formate sunt larg acceptate de bazele de date spațiale și de bibliotecile de procesare geografică, ceea ce asigură interoperabilitatea între componentele sistemului.

Folosirea unor formate standardizate de reprezentare a geometriilor este importantă pentru extensibilitatea aplicației: datele pot fi importate din surse externe, prelucrate cu instrumente specializate și apoi consumate de aplicație, fără a fi nevoie de conversii proprietare. Acest aspect susține unul dintre obiectivele declarate ale proiectului, și anume capacitatea de a adăuga ușor noi clădiri și zone fără modificări majore în logica aplicației.

1.3.3 OpenStreetMap ca sursă de date

OpenStreetMap (OSM) este un proiect colaborativ care pune la dispoziție date geografice deschise, contribuite și întreținute de o comunitate largă de utilizatori din întreaga lume. Datele OSM acoperă o gamă variată de elemente: drumuri, clădiri, alei pietonale, puncte de interes și multe altele. Faptul că aceste date sunt deschise și pot fi descărcate și prelucrate liber le face o sursă valoroasă pentru aplicațiile de navigare, mai ales în contextul în care construirea de la zero a unui set complet de date geografice ar fi extrem de costisitoare.

În cadrul ETTIway, datele provenite din OpenStreetMap sunt folosite pentru reprezentarea infrastructurii exterioare a campusului, în special pentru afișarea drumurilor și a aleilor. Modelul de date OSM organizează informația în jurul a trei tipuri de elemente fundamentale: noduri, care reprezintă puncte individuale definite prin coordonate; căi, care reprezintă secvențe ordonate de noduri și pot descrie atât linii, cât și conturul unor suprafețe; și relații, care grupează mai multe elemente pentru a descrie structuri complexe. Fiecărui element i se pot asocia etichete sub formă de perechi cheie-valoare, care îi descriu semnificația, de exemplu tipul de drum sau funcția unei clădiri. Această structură flexibilă permite filtrarea și selecția elementelor relevante pentru navigarea în campus. Datele extrase din OpenStreetMap sunt adaptate scenariilor de navigare din campus, fiind reținute doar elementele relevante — drumuri, alei și contururi de clădiri — și eliminate informațiile care nu prezintă interes pentru aplicație. Organizarea atentă a stratului de date reprezintă, de altfel, o parte semnificativă a contribuției proprii descrise în lucrare.

1.4 Poziționarea și localizarea utilizatorului

Pe lângă reprezentarea statică a infrastructurii campusului, un sistem de navigare complet trebuie să poată determina poziția curentă a utilizatorului și să o raporteze la harta afișată. Această capacitate transformă harta dintr-un simplu instrument de consultare într-un asistent de navigare contextual, capabil să indice nu doar unde se află o destinație, ci și cum poate fi atinsă pornind din locația curentă.

1.4.1 Sistemul de poziționare globală

Determinarea poziției în spațiul exterior se bazează, în mod tipic, pe sistemul de poziționare globală (GPS), o infrastructură de sateliți care permite unui receptor să își calculeze coordonatele pe baza semnalelor recepționate. Fiecare satelit transmite continuu informații privind poziția sa și momentul exact al emisiei; receptorul măsoară diferențele de timp dintre semnalele provenite de la mai mulți sateliți și, prin triangulație, deduce propria poziție în coordonate de latitudine și longitudine. Pentru o estimare bidimensională a poziției sunt necesare semnale de la cel puțin trei sateliți, iar pentru includerea altitudinii este necesar un al patrulea.

În aplicațiile web, accesul la poziția dispozitivului nu se realizează direct cu hardware-ul GPS, ci printr-o interfață standardizată pusă la dispoziție de browser, care abstractizează sursa concretă a informației de localizare. Browserul poate combina mai multe surse — semnalul GPS propriu-zis, rețelele de telefonie mobilă și punctele de acces de tip Wi-Fi — pentru a oferi o estimare a poziției chiar și atunci când semnalul satelitar este slab. Această interfață furnizează, pe lângă coordonatele propriu-zise, și o estimare a preciziei, exprimată ca o rază de incertitudine în jurul poziției raportate.

1.4.2 Precizia și limitările localizării

Precizia localizării variază semnificativ în funcție de mediu și de sursele disponibile. În spațiu deschis, cu vizibilitate directă către sateliți, eroarea tipică este de ordinul câtorva metri. În schimb, în interiorul clădirilor sau în apropierea structurilor înalte, semnalul satelitar este atenuat ori reflectat, iar precizia se degradează considerabil, putând ajunge la zeci de metri. Acest fenomen este deosebit de relevant pentru un campus universitar, unde clădirile sunt dese și utilizatorii petrec mult timp în interior — exact contextul în care poziționarea satelitară devine cel mai puțin fiabilă.

Din această cauză, un sistem de navigare robust trebuie să trateze explicit situațiile de imprecizie sau de lipsă temporară a semnalului, evitând afișarea unei poziții înșelător de exacte și informând utilizatorul cu privire la gradul de încredere al localizării. În cadrul ETTIway, poziția utilizatorului este folosită ca punct de plecare pentru calculul traseului, fiind asociată celui mai apropiat punct de pe rețeaua de trasee a campusului, astfel încât micile erori de localizare să nu împiedice funcționarea rutării.

1.5 Algoritmi de determinare a drumului minim

Componenta de rutare reprezintă nucleul funcțional al oricărui sistem de navigare. Odată ce infrastructura campusului este reprezentată sub forma unui graf, problema găsirii celui mai scurt traseu între două puncte devine o problemă clasică de teoria grafurilor: determinarea drumului de cost minim între două noduri. Această secțiune prezintă fundamentele teoretice ale problemei și algoritmi care o rezolvă, cu accent pe soluția folosită în implementare.

1.5.1 Modelarea problemei prin grafuri

Un graf este o structură formată dintr-un set de noduri și un set de muchii care leagă perechi de noduri. În contextul navigării, nodurile corespund punctelor de decizie — intersecții, intrări, capete de alei — iar muchiile corespund segmentelor de traseu care le unesc. Fiecărei muchii i se asociază o pondere, care exprimă costul parcurgerii segmentului respectiv; în cazul cel mai simplu, ponderea este distanța geografică reală dintre cele două noduri, dar pot fi folosite și alte criterii, precum timpul estimat de parcurgere sau gradul de accesibilitate.

Atunci când ponderile reprezintă distanțe, graful este nedirecționat, deoarece o alee poate fi parcursă în ambele sensuri cu același cost. Problema centrală devine astfel găsirea, în acest graf ponderat, a lanțului de muchii care leagă nodul de start de nodul de destinație și a cărui sumă a ponderilor este minimă. Această formulare permite aplicarea unor algoritmi consacrați, studiați intens în literatura de specialitate [?].

1.5.2 Algoritmul lui Dijkstra

Cel mai cunoscut algoritm pentru determinarea drumului minim de la un nod sursă către celelalte noduri ale unui graf cu ponderi nenegative este algoritmul lui Dijkstra. Principiul său de funcționare este unul de tip lacom (*greedy*): algoritmul menține, pentru fiecare nod, cea mai mică distanță cunoscută până la acel moment față de nodul sursă, precum și nodul precedent pe traseul optim. La fiecare pas, este selectat nodul nevizitat cu distanța minimă, după care se încearcă îmbunătățirea distanțelor către vecinii săi — operație cunoscută sub numele de relaxare a muchiilor. Procesul continuă până când nodul de destinație este atins sau până când toate nodurile accesibile au fost vizitate.

Corectitudinea algoritmului se bazează pe observația că, atunci când un nod este selectat ca având distanța minimă dintre cele nevizitate, această distanță este deja optimă și nu mai poate fi îmbunătățită ulterior, întrucât toate ponderile sunt nenegative. La final, traseul optim se reconstruiește pornind de la nodul de destinație și urmărind, în sens invers, lanțul nodurilor precedente până la sursă. Acest mod de funcționare îl face potrivit pentru calculul traseelor într-o rețea de dimensiuni moderate, cum este cea a unui campus universitar.

1.5.3 Performanță și optimizări

Performanța algoritmului lui Dijkstra depinde în mod esențial de modul în care este implementată operația de selectare a nodului cu distanța minimă. Într-o implementare directă, în care la fiecare pas se parcurg toate nodurile pentru a-l găsi pe cel cu distanța minimă, complexitatea devine pătratică în raport cu numărul de noduri. Pentru grafuri mari, această abordare devine ineficientă, iar literatura de specialitate propune optimizări bazate pe structuri de date de tip coadă de priorități, care reduc semnificativ timpul de execuție prin extragerea rapidă a nodului minim [?].

Pentru rețeaua unui campus universitar, în care numărul de noduri este relativ redus, implementarea directă a algoritmului oferă timpi de răspuns acceptabili, calculul traseului fiind perceput ca instantaneu de către utilizator. Tocmai din acest motiv, în cadrul ETTIway s-a optat pentru o implementare clară și ușor de urmărit a algoritmului, prioritizând lizibilitatea codului în detrimentul unor optimizări care nu ar fi adus beneficii practice la această scară. Aceleași principii stau la baza serviciilor de rutare la scară largă construite peste date deschise precum cele din OpenStreetMap [?], care însă necesită structuri de date și optimizări mult mai complexe pentru a trata rețele de dimensiunea unor orașe întregi.

1.6 Interfețe pentru dispozitive mobile

Un sistem de navigare în campus este utilizat, în mod natural, preponderent de pe dispozitive mobile: utilizatorul are nevoie de orientare tocmai atunci când se deplasează prin campus, telefonul fiind dispozitivul aflat permanent la îndemână. Din acest motiv, proiectarea interfeței

trebuie să țină cont, încă de la început, de constrângerile specifice ecranelor de mici dimensiuni și ale interacțiunii tactile.

Principiul de proiectare adecvat acestui context este cel al designului responsiv, prin care interfața se adaptează automat la dimensiunea ecranului pe care este afișată, reorganizând elementele pentru a rămâne lizibile și utilizabile atât pe un monitor de calculator, cât și pe ecranul unui telefon. În cazul unei aplicații de hărți, acest lucru presupune ca harta să ocupe cât mai mult din suprafața disponibilă, iar elementele de control — căutarea, lista de clădiri, detaliile — să fie organizate astfel încât să nu obtureze harta, putând fi afișate sau ascunse după nevoie.

Pe lângă adaptarea vizuală, interacțiunea tactilă impune constrângeri proprii: elementele care pot fi atinse trebuie să fie suficient de mari pentru a fi acționate cu degetul, iar gesturile uzuale — deplasarea hărții prin glisare, mărirea prin apropierea degetelor — trebuie să funcționeze natural. Biblioteca folosită pentru afișarea hărții în ETTIway oferă suport nativ pentru aceste gesturi, ceea ce simplifică obținerea unei experiențe fluide pe dispozitivele mobile, în acord cu obiectivul declarat al proiectului de a oferi o soluție accesibilă și ușor de utilizat în deplasare.

1.7 Arhitecturi de aplicații web

Platforma ETTIway este construită pe o arhitectură de tip client–server, în care responsabilitățile sunt separate clar între componenta care rulează în browserul utilizatorului și componenta care rulează pe server. Această separare este o practică consacrată în dezvoltarea aplicațiilor web moderne și aduce beneficii importante în privința mentenabilității și a extensibilității.

1.7.1 Separarea frontend–backend

Frontend-ul, adică partea aplicației care rulează în browserul utilizatorului, este responsabil de prezentarea informației și de interacțiunea directă cu utilizatorul: afișarea hărții, gestionarea evenimentelor de tip clic, încărcarea planurilor de etaj și prezentarea detaliilor despre clădiri. Backend-ul, adică partea care rulează pe server, este responsabil de gestionarea datelor, de expunerea serviciilor prin care frontend-ul obține informația și de aplicarea logicii de prelucrare necesare.

Această separare a responsabilităților permite dezvoltarea și testarea independentă a celor două componente. Frontend-ul poate fi modificat fără a afecta logica de date, iar backend-ul poate fi extins cu noi servicii fără a necesita rescrierea interfeței. Comunicarea dintre cele două straturi se realizează printr-o interfață bine definită de servicii, ceea ce reduce cuplarea dintre componente și facilitează evoluția independentă a acestora.

1.7.2 Servicii REST și formatul JSON

Comunicarea dintre frontend și backend în ETTIway se realizează prin servicii de tip REST (*Representational State Transfer*). Acest stil arhitectural se bazează pe protocolul HTTP și organizează accesul la date în jurul noțiunii de resurse, fiecare resursă fiind identificată printr-o adresă și manipulată prin operațiile standard ale protocolului. Avantajul acestui model constă în simplitatea și în caracterul său larg adoptat: un serviciu REST poate fi consumat de orice client capabil să efectueze cereri HTTP, indiferent de tehnologia pe care este construit.

Pentru schimbul efectiv de date se folosește formatul JSON (*JavaScript Object Notation*), un format text ușor de citit atât de oameni, cât și de mașini, care permite reprezentarea structurată a obiectelor și a colecțiilor. JSON s-a impus ca format standard în comunicarea dintre aplicațiile web datorită simplității sale și a suportului nativ existent în limbajul JavaScript, folosit pentru dezvoltarea frontend-ului. În cadrul aplicației, datele despre clădiri, etaje și puncte de interes sunt transmise între server și client sub formă de obiecte JSON.

1.8 Tehnologii utilizate în implementare

În această secțiune sunt prezentate tehnologiile concrete pe care se sprijină implementarea platformei ETTIway. Alegerea acestor tehnologii a fost ghidată de mai multe criterii: maturitatea și stabilitatea soluțiilor, disponibilitatea unei documentații cuprinzătoare, caracterul deschis al instrumentelor și potrivirea cu cerințele specifice ale unui sistem de navigare în campus.

1.8.1 Spring Boot pentru backend

Backend-ul aplicației este realizat cu Spring Boot, un cadru de lucru (*framework*) pentru limbajul Java, destinat dezvoltării rapide de aplicații și servicii web. Spring Boot reduce semnificativ efortul de configurare necesar pornirii unui proiect, oferind un set de convenții și de componente preconfigurate care permit dezvoltatorului să se concentreze asupra logicii aplicației, nu asupra detaliilor de infrastructură. Printre avantajele sale se numără un server web încorporat, un sistem robust de gestiune a dependențelor și o integrare facilă cu bazele de date.

În cadrul ETTIway, Spring Boot este folosit pentru a expune serviciile REST prin care frontend-ul obține datele despre clădiri și despre infrastructura campusului, precum și pentru a gestiona accesul la stratul de date. Pentru interacțiunea cu baza de date se utilizează mecanisme de persistență de tip JPA (*Java Persistence API*), care permit maparea obiectelor din aplicație pe înregistrările din baza de date, reducând cantitatea de cod necesar pentru operațiile uzuale de acces la date.

1.8.2 Leaflet și JavaScript pentru frontend

Interfața de vizualizare a hărții este construită folosind biblioteca Leaflet, o bibliotecă JavaScript deschisă, dedicată afișării hărților interactive în browser. Leaflet a fost aleasă datorită dimensiunii sale reduse, a performanței bune și a simplității interfeței de programare. Biblioteca oferă suport nativ pentru afișarea hărților bazate pe dale, pentru gestionarea nivelurilor de zoom, pentru adăugarea de markere și pentru organizarea informației pe straturi distincte care pot fi afișate sau ascunse independent.

Restul logicii de pe partea de client este realizat în JavaScript, fără a apela la un cadru de lucru complex pentru interfață. Această alegere a fost motivată de dorința de a păstra aplicația ușoară și de a reduce dependențele externe, în acord cu obiectivul de a obține timpi de răspuns reduși și o experiență fluidă. Interfața este structurată într-un panou lateral, care găzduiește funcțiile de căutare, lista clădirilor disponibile și detaliile clădirii selectate, și o zonă principală ocupată de harta interactivă. Vizualizarea planurilor de etaj se realizează printr-o fereastră modală, care permite parcurgerea nivelurilor unei clădiri fără a părăsi contextul hărții.

1.8.3 Organizarea datelor și extensibilitatea

Datele despre clădiri și despre structura lor interioară sunt organizate într-o formă structurată, care asociază fiecărei clădiri un identificator, denumirea, poziția geografică, o descriere și lista nivelurilor sale, fiecare nivel având asociat un plan corespunzător. Această organizare a fost gândită pentru a fi ușor de extins: adăugarea unei noi clădiri sau a unui nou etaj presupune completarea structurii de date, fără a fi necesare modificări în logica aplicației. Extensibilitatea reprezintă un obiectiv explicit al proiectului, deoarece un campus universitar este un mediu dinamic, în care infrastructura se modifică în timp, iar o soluție viabilă trebuie să poată absorbi aceste schimbări cu efort minim.

1.9 Concluziile capitolului

În acest capitol au fost prezentate fundamentele teoretice și tehnologice care stau la baza platformei ETTIway. Au fost introduse conceptele de sistem de navigare digitală și de hartă interactivă, distincția dintre navigarea exterioară și cea interioară, precum și provocarea integrării acestora într-un flux unitar. S-au discutat apoi conceptele esențiale legate de datele geospațiale — sisteme de referință, reprezentarea geometriilor și sursele de date deschise. A fost analizat mecanismul de poziționare a utilizatorului prin sistemul GPS, împreună cu limitările sale de precizie în mediul construit, și au fost prezentate fundamentele algoritmilor de determinare a drumului minim, cu accent pe algoritmul lui Dijkstra, folosit pentru calculul traseelor. În final, au fost discutate cerințele de proiectare a interfețelor pentru dispozitive mobile și tehnologiile concrete utilizate în implementare, de la cadrul Spring Boot pentru backend până la biblioteca

Leaflet pentru vizualizarea hărții, subliniind în fiecare caz motivele care au stat la baza alegerii lor.

Înțelegerea acestor elemente este necesară pentru parcurgerea capitolelor următoare, în care vor fi prezentate analiza cerințelor, arhitectura detaliată a sistemului și modul concret de implementare a funcționalităților platformei. Plecând de la fundamentele expuse aici, următorul capitol abordează cerințele funcționale și nefuncționale ale aplicației, precum și deciziile de proiectare care decurg din acestea.

Capitolul 2

Arhitectura sistemului

Acest capitol prezintă arhitectura platformei ETTIway, plecând de la o vedere de ansamblu asupra componentelor și ajungând la detalierea fiecărui nivel al sistemului. Sunt descrise separarea responsabilităților între frontend și backend, structura datelor persistate, mecanismul de autentificare și autorizare, precum și fluxurile principale prin care componentele comunică între ele. Deciziile de proiectare sunt motivate prin raportare la cerințele formulate în capitolul anterior, cu accent pe extensibilitate, securitate și claritate a fluxului de utilizare.

2.1 Vedere de ansamblu asupra arhitecturii

Platforma ETTIway este construită pe o arhitectură de tip client-server, organizată pe trei niveluri logice distincte. Primul nivel este cel de prezentare (frontend), realizat în HTML, CSS și JavaScript, care rulează în browserul utilizatorului și se ocupă de afișarea hărții interactive, de interacțiunea cu utilizatorul și de calculul traseelor. Al doilea nivel este cel al serviciilor (backend), realizat cu Spring Boot, care expune servicii web, gestionează autentificarea și autorizarea utilizatorilor și mediază accesul la date. Al treilea nivel este cel al datelor, reprezentat de o bază de date relațională PostgreSQL, în care sunt stocate atât conturile utilizatorilor, cât și informația despre structura hărții.

Separarea pe aceste trei niveluri aduce mai multe avantaje. În primul rând, permite dezvoltarea și testarea independentă a fiecărei componente, deoarece interfața dintre niveluri este bine definită. În al doilea rând, izolează logica de securitate la nivelul backend-ului, astfel încât regulile de acces nu pot fi ocolite din interfața clientului. În al treilea rând, facilitează extinderea sistemului: adăugarea de noi servicii sau de noi tipuri de date nu necesită rescrierea componentelor existente, ci doar completarea celor noi.

Comunicarea dintre nivelul de prezentare și cel al serviciilor se realizează prin cereri HTTP către un set de servicii de tip REST, datele fiind transferate în format JSON. Această alegere asigură un cuplaj redus între client și server: orice modificare internă a backend-ului rămâne transparentă pentru frontend, atât timp cât contractul serviciilor este respectat. În secțiunile următoare, fiecare nivel este analizat în detaliu.

2.2 Nivelul de servicii (backend)

Backend-ul aplicației este o aplicație Spring Boot, structurată pe pachete distincte în funcție de responsabilități. Pachetul de configurare conține clasele de inițializare, de securitate și de pornire a aplicației; pachetul de entități conține clasele care modelează datele persistate; pachetul de repository conține interfețele de acces la baza de date. Această organizare urmează convențiile consacrate ale ecosistemului Spring și face codul ușor de parcurs și de extins.

2.2.1 Punctul de intrare al aplicației

Pornirea aplicației este gestionată de clasa principală, adnotată cu `@SpringBootApplication`. Această adnotare activează mecanismele de configurare automată ale Spring Boot, prin care componentele sunt descoperite și inițializate fără configurare manuală explicită. Codul punctului de intrare este minimal, întreaga logică de configurare fiind delegată claselor specializate.

```
1 @SpringBootApplication
2 public class DemoApplication {
3
4     public static void main(String[] args) {
5         SpringApplication.run(DemoApplication.class, args);
6     }
7 }
```

Listing 2.1: Clasa principală a aplicației Spring Boot

La pornire, pe lângă inițializarea contextului Spring, aplicația execută și o componentă de inițializare a datelor, descrisă în secțiunea dedicată autentificării, care asigură existența unui cont de administrator în baza de date.

2.2.2 Accesul la date prin JPA

Accesul la baza de date se realizează prin intermediul JPA (*Java Persistence API*), folosind abstracțiile oferite de Spring Data. În locul scrierii manuale a interogărilor, accesul la date este definit prin interfețe care extind `JpaRepository`, Spring generând automat implementările necesare. Pentru entitatea utilizator, repository-ul definește în plus o metodă de căutare după numele de utilizator, al cărei corp este generat automat pe baza denumirii metodei.

```
1 public interface UserRepository extends JpaRepository<User, Long> {
2     Optional<User> findByUsername(String username);
3 }
```

Listing 2.2: Interfața de acces la date pentru utilizatori

Avantajul acestei abordări este reducerea semnificativă a codului repetitiv: operațiile uzuale de salvare, căutare și ștergere sunt moștenite din `JpaRepository`, iar metodele specifice se declară prin simpla lor semnătură. Tipul de retur `Optional` exprimă explicit faptul că un

utilizator căutat ar putea să nu existe, obligând codul apelant să trateze acest caz și reducând astfel riscul de erori.

2.3 Nivelul de date

Datele persistate sunt stocate într-o bază de date relațională PostgreSQL, aleasă pentru maturitatea sa, pentru suportul robust al tipurilor de date și pentru caracterul său deschis. Maparea dintre tabelele bazei de date și obiectele din aplicație este realizată prin adnotări JPA pe clasele de tip entitate. În cadrul ETTIway există două entități principale: utilizatorii și datele hărții.

2.3.1 Entitatea utilizator

Entitatea `User` modelează un cont din sistem și este mapată pe tabelul `users`. Fiecare utilizator are un identificator generat automat, un nume de utilizator unic, o parolă stocată în formă criptată și un rol care determină nivelul de acces. Constrângerile de unicitate și de obligativitate sunt exprimate direct prin adnotări la nivel de coloană.

```
1 @Entity
2 @Table(name = "users")
3 public class User {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @Column(unique = true, nullable = false)
9     private String username;
10
11     @Column(nullable = false)
12     private String password;
13
14     private String role; // de exemplu "ROLE_ADMIN"
15
16     public User() {}
17
18     // metode de tip getter si setter
19 }
```

Listing 2.3: Entitatea `User` mapată pe tabelul `users`

Constructorul fără argumente este necesar pentru ca JPA să poată instanția obiectul atunci când citește o înregistrare din baza de date, iar metodele de tip `getter` și `setter` permit citirea și scrierea câmpurilor. Parola nu este niciodată stocată în clar, ci doar în forma sa criptată, aspect detaliat în secțiunea despre securitate.

2.3.2 Entitatea pentru datele hărții

A doua entitate, `MapData`, este responsabilă de persistarea structurii hărții campusului. Pentru a permite o reprezentare flexibilă a grafului de noduri și muchii pe care se bazează rutarea, structura este serializată în format JSON și stocată într-o coloană de tip text. Această abordare evită rigidizarea schemei bazei de date și permite modificarea structurii grafului fără a fi necesare migrări ale schemei relaționale.

```
1 @Entity
2 @Table(name = "map_data")
3 public class MapData {
4
5     @Id
6     private Long id;
7
8     @Column(columnDefinition = "TEXT")
9     private String graphJson;
10
11     private LocalDateTime updatedAt;
12
13     public MapData() {}
14
15     // metode de tip getter si setter
16 }
```

Listing 2.4: Entitatea `MapData` pentru stocarea structurii hărții

Câmpul `graphJson` conține reprezentarea serializată a grafului, iar câmpul `updatedAt` reține momentul ultimei actualizări, util pentru urmărirea modificărilor aduse hărții. Stocarea grafului ca document JSON constituie o decizie de proiectare orientată spre flexibilitate: nodurile și muchiile pot fi adăugate sau modificate fără a impune o structură fixă la nivelul tabelor.

2.4 Securitate: autentificare și autorizare

Un aspect central al arhitecturii ETTIway îl constituie mecanismul de securitate, care diferențiază între utilizatorii obișnuiți și administratori. Această diferențiere este esențială, deoarece administratorii pot modifica structura hărții, în timp ce utilizatorii obișnuiți au acces doar în mod de vizualizare. Securitatea este implementată folosind Spring Security, integrat nativ în ecosistemul Spring Boot.

2.4.1 Criptarea parolelor

Parolele utilizatorilor nu sunt stocate în clar în baza de date, ci sunt criptate folosind algoritmul BCrypt, prin intermediul unei componente dedicate de codare a parolelor. BCrypt este un algoritm de tip funcție de derivare a cheii, conceput special pentru stocarea parolelor: el este

lent în mod intenționat și încorporează un factor aleatoriu, ceea ce îngreunează semnificativ atacurile prin forță brută.

```
1 @Bean
2 public PasswordEncoder passwordEncoder() {
3     return new BCryptPasswordEncoder();
4 }
```

Listing 2.5: Definirea componentei de criptare a parolelor

Componenta de criptare este definită ca un *bean* Spring, ceea ce permite injectarea sa automată în orice altă componentă care are nevoie de criptarea sau verificarea parolelor. La crearea unui cont, parola este trecută prin această componentă înainte de a fi salvată, iar la autentificare, parola introdusă este comparată cu cea criptată din baza de date.

2.4.2 Configurarea regulilor de acces

Regulile care stabilesc ce resurse sunt accesibile public, ce resurse necesită autentificare și ce resurse sunt rezervate administratorilor sunt definite într-un lanț de filtre de securitate. Resursele statice și paginile de autentificare sunt publice, rutele administrative sunt rezervate utilizatorilor cu rol de administrator, iar restul paginilor necesită cel puțin autentificare.

```
1 .authorizeHttpRequests(auth -> auth
2     // resurse publice
3     .requestMatchers("/login.html", "/api/auth/**",
4         "/css/**", "/js/**", "/data/**").permitAll()
5     // rute rezervate administratorului
6     .requestMatchers("/api/admin/**").hasRole("ADMIN")
7     // restul cere autentificare
8     .anyRequest().authenticated()
9 )
```

Listing 2.6: Definirea regulilor de autorizare

Autentificarea propriu-zisă se realizează printr-un formular de login, configurat să folosească o pagină proprie și să redirecționeze utilizatorul, după succes, către o rută care verifică rolul acestuia și îl direcționează corespunzător. Deconectarea invalidează sesiunea și readuce utilizatorul la pagina de autentificare.

2.4.3 Încărcarea utilizatorilor și inițializarea contului de administrator

Pentru ca Spring Security să poată verifica identitatea unui utilizator, este necesară o componentă care să încarce datele acestuia din baza de date pe baza numelui de utilizator. Această componentă implementează interfața `UserDetailsService` și transformă entitatea proprie `User` într-un obiect pe care mecanismul de securitate îl poate folosi, inclusiv rolul asociat.

```

1 @Override
2 public UserDetails loadUserByUsername(String username) {
3     User user = userRepository.findByUsername(username)
4         .orElseThrow(() -> new UsernameNotFoundException("User not found"));
5
6     return org.springframework.security.core.userdetails.User
7         .withUsername(user.getUsername())
8         .password(user.getPassword())
9         .authorities(new SimpleGrantedAuthority(user.getRole()))
10        .build();
11 }

```

Listing 2.7: Încărcarea utilizatorului pentru autentificare

Pentru a asigura existența unui cont de administrator încă de la prima pornire a aplicației, este folosită o componentă de inițializare care se execută automat la lansare. Aceasta verifică dacă există deja un cont de administrator și, în caz contrar, îl creează, criptând în prealabil parola. Astfel, sistemul devine utilizabil imediat după instalare, fără a necesita o configurare manuală a bazei de date.

```

1 @Bean
2 CommandLineRunner init(UserRepository userRepository,
3     PasswordEncoder passwordEncoder) {
4     return args -> {
5         if (userRepository.findByUsername("admin").isEmpty()) {
6             User admin = new User();
7             admin.setUsername("admin");
8             admin.setPassword(passwordEncoder.encode("admin123"));
9             admin.setRole("ROLE_ADMIN");
10            userRepository.save(admin);
11        }
12    };
13 }

```

Listing 2.8: Inițializarea automată a contului de administrator

Acest mecanism de inițializare ilustrează un principiu de proiectare orientat spre ușurința punerii în funcțiune: starea minimă necesară funcționării sistemului este creată automat, reducând efortul de configurare și posibilitatea unor erori manuale.

2.5 Nivelul de prezentare (frontend)

Nivelul de prezentare este realizat în JavaScript, folosind biblioteca Leaflet pentru afișarea hărții interactive. Pe lângă afișarea propriu-zisă, frontend-ul încorporează și o parte importantă din logica aplicației, în special construirea grafului de navigare și calculul traseelor. Această secțiune descrie elementele principale ale interfeței și mecanismele de prelucrare implementate pe partea de client.

2.5.1 Inițializarea hărții și delimitarea zonei

Harta este centrată pe coordonatele campusului și limitată la o zonă geografică bine definită, pentru a împiedica utilizatorul să navigheze în afara regiunii de interes. Coordonatele de centrare și limitele zonei sunt definite ca valori de configurare, ceea ce permite adaptarea facilă a aplicației la un alt campus prin simpla modificare a acestor valori.

```
1 const DEFAULT_CAMPUS_LAT = 44.433215;
2 const DEFAULT_CAMPUS_LON = 26.056764;
3 const DEFAULT_ZOOM_LEVEL = 18;
4 const MAP_BOUNDS = [
5     [44.432102, 26.054500], // coltul de sud-vest
6     [44.434329, 26.059028] // coltul de nord-est
7 ];
```

Listing 2.9: Parametrii de configurare ai hărții

Pe lângă afișarea hărții, interfața include un panou lateral care găzduiește funcțiile de căutare, lista clădirilor disponibile și detaliile clădirii sau ale sălii selectate. Detaliile despre o sală — clădirea, etajul, capacitatea și tipul — sunt generate dinamic, cu prelucrarea atentă a textului pentru a preveni problemele de afișare și eventualele riscuri de securitate, prin transformarea caracterelor speciale înainte de inserarea în pagină.

2.5.2 Construirea grafului de navigare

Pentru a permite calculul traseelor între puncte, infrastructura campusului este transformată într-un graf, în care nodurile reprezintă intersecții sau puncte de pe trasee, iar muchiile reprezintă segmentele care le leagă. Graful este construit pornind de la datele geografice exprimate în format GeoJSON, în care elementele de tip linie descriu aleile și drumurile.

Fiecare nod este identificat printr-o cheie obținută din coordonatele sale, rotunjite la o precizie fixă, pentru a asigura că puncte foarte apropiate sunt tratate ca un singur nod. Pentru fiecare segment al unei linii se adaugă o muchie între nodurile adiacente, ponderea muchiei fiind distanța geografică reală dintre cele două puncte. Graful rezultat este nedirecționat, deoarece o alee poate fi parcursă în ambele sensuri.

```
1 function buildGraphFromGeoJSON(geoJSON) {
2     const graph = {};
3     const nodesMap = {};
4
5     function addNode(lat, lng) {
6         const key = getCoordKey(lat, lng);
7         if (!graph[key]) {
8             graph[key] = [];
9             nodesMap[key] = L.latLng(lat, lng);
10        }
11        return key;
```

```

12     }
13     function addEdge(key1, key2, dist) {
14         graph[key1].push({ node: key2, weight: dist });
15         graph[key2].push({ node: key1, weight: dist });
16     }
17     // parcurgerea elementelor de tip linie din GeoJSON
18     // si adaugarea nodurilor si muchiilor corespunzatoare
19     return { graph, nodesMap };
20 }

```

Listing 2.10: Construirea grafului din date GeoJSON

Observația importantă privind formatul GeoJSON este că acesta exprimă coordonatele în ordinea longitudine–latitudine, în timp ce Leaflet le așteaptă în ordinea latitudine–longitudine; conversia corectă între cele două convenții este necesară pentru ca punctele să fie plasate corect pe hartă.

2.5.3 Calculul traseului prin algoritmul lui Dijkstra

Determinarea celui mai scurt traseu între două puncte se realizează prin algoritmul lui Dijkstra, implementat direct pe partea de client. Plecând de la nodul de start, algoritmul menține pentru fiecare nod distanța minimă cunoscută și nodul precedent pe traseul optim, alegând la fiecare pas nodul nevizitat cu distanța cea mai mică și actualizând distanțele vecinilor săi. Atunci când nodul de destinație este atins, traseul este reconstruit parcurgând în sens invers lanțul nodurilor precedente.

```

1 function runDijkstra(graph, nodesMap, startKey, endKey) {
2     const distances = {};
3     const previous = {};
4     const unvisited = new Set();
5
6     for (const node in graph) {
7         distances[node] = Infinity;
8         previous[node] = null;
9         unvisited.add(node);
10    }
11    distances[startKey] = 0;
12    // se alege repetat nodul nevizitat cu distanta minima
13    // si se actualizeaza distantele vecinilor
14    // ...
15    // reconstructia traseului din lantul de noduri precedente
16 }

```

Listing 2.11: Nucleul algoritmului lui Dijkstra

Deoarece punctele alese de utilizator nu coincid, în general, exact cu nodurile grafului, este necesară o etapă de asociere a fiecărui punct cu cel mai apropiat nod existent. Aceasta se

realizează printr-o căutare a nodului aflat la distanța minimă față de punctul țintă, asigurând că rutarea pornește și se încheie în puncte aflate efectiv pe rețeaua de trasee.

```
1 function findNearestNode(targetLatLng, nodesMap) {  
2     let nearestKey = null;  
3     let minDistance = Infinity;  
4     for (const key in nodesMap) {  
5         const dist = targetLatLng.distanceTo(nodesMap[key]);  
6         if (dist < minDistance) {  
7             minDistance = dist;  
8             nearestKey = key;  
9         }  
10    }  
11    return nearestKey;  
12 }
```

Listing 2.12: Găsirea celui mai apropiat nod din graf

Dacă între cele două puncte nu există niciun traseu — de exemplu, atunci când o porțiune a rețelei este izolată — algoritmul returnează o valoare care semnalează absența unei rute, iar interfața poate informa utilizatorul în consecință.

2.6 Fluxul de comunicare între componente

Reunind cele trei niveluri, fluxul tipic de utilizare al aplicației poate fi descris astfel. La accesarea aplicației, utilizatorul este direcționat către pagina de autentificare, iar după validarea credențialelor de către backend, rolul său determină modul de funcționare: vizualizare pentru utilizatorii obișnuiți, respectiv editare pentru administratori. Frontend-ul solicită apoi structura hărții de la backend, care o extrage din baza de date și o transmite în format JSON. Pe baza acestor date, clientul construiește graful de navigare și afișează harta interactivă.

Atunci când utilizatorul selectează un punct de plecare și unul de destinație, calculul traseului se realizează integral pe partea de client, prin construirea grafului, asocierea punctelor cu cele mai apropiate noduri și aplicarea algoritmului lui Dijkstra. Această alegere reduce sarcina serverului și asigură un timp de răspuns redus, traseul fiind calculat și afișat fără a necesita o nouă cerere către backend. Modificările aduse structurii hărții de către un administrator sunt, în schimb, transmise către backend și persistate în baza de date, devenind astfel disponibile tuturor utilizatorilor la accesările ulterioare.

2.7 Concluziile capitolului

În acest capitol a fost prezentată arhitectura platformei ETTIway, organizată pe trei niveluri: prezentare, servicii și date. A fost descris backend-ul realizat cu Spring Boot, cu accesul la date prin JPA și cu persistarea în PostgreSQL, precum și cele două entități principale ale sistemului.

S-a detaliat mecanismul de securitate, bazat pe Spring Security, cu criptarea parolelor prin BCrypt, diferențierea rolurilor și inițializarea automată a contului de administrator. La nivelul de prezentare au fost prezentate inițializarea hărții, construirea grafului de navigare din date GeoJSON și implementarea algoritmului lui Dijkstra pentru calculul traseelor.

Deciziile de proiectare descrise — separarea pe niveluri, stocarea flexibilă a grafului, calculul traseelor pe partea de client și inițializarea automată a sistemului — au fost orientate spre extensibilitate, securitate și ușurința punerii în funcțiune. Capitolul următor abordează modul concret de utilizare a aplicației și rezultatele obținute, ilustrând funcționarea sistemului în scenariile reale de navigare din campus.